8253jang@daum.net
장희정

SpringBoot기반의 Restful API와 React 연결하기

주요기능

- **회원관리(아이디, 비밀번호, 이름, 주소, role, 등록일)**
  - 회원가입(아이디중복체크)
  - 로그인
  - 로그아웃

- **게시판관리(글번호, 제목, 내용, 작성자, 등록일, 수정일)**
  - 게시물 전체조회 – 아무나 조회
  - 게시물 상세보기 – 인증된 사용자 상세보기
  - 게시물등록 – 인증된 사용자 만이 등록가능
  - 게시물수정 - 인증된 사용자 + 자신이 쓴 글인 경우만
  - 게시물 삭제- 인증된 사용자 + 자신이 쓴 글인 경우만

주요기능 만들기

- 회원가입(JoinForm.jsx파일)

☞ **가입정보에 대한 상태관리를 위해서 useState를 선언한다.**
회원가입의 속성들을 하나의 객체로 관리한다.

```
const [member, setMember] = useState({
    id :'' ,
    name:'',
    pwd:'',
    address:''
  });
```

☞ **input 요소의 value가 변경되었을 때 setMember를 이용하여 변경한다.**

```
  const changeValue = (e)=>{
      //console.log(e.target.name);
     //console.log(e.target.value);
      setMember({
        ...member,
```

```
            [e.target.name]:e.target.value}
          )
}
```

☞ 아이디 중복체크 하기

Axios를 이용하여 backEnd에 요청을 보내고 그 결과를 받는다.

```
if(e.target.name==="id"&&e.target.value!==""){
        axios({
          method:"GET",
          url :"http://localhost:9000/members/"+e.target.value,
        })
        .then((res)=>{
            console.log(res);
        })
        .catch((err)=>{
         //실패
          leterrMessage = err.response.data.type +"₩n";
          errMessage +=  err.response.data.title +"₩n";
          errMessage +=  err.response.data.detail +"₩n";
          errMessage +=  err.response.data.status +"₩n";
          errMessage +=  err.response.data.instance +"₩n";
          errMessage +=  err.response.data.timestamp;
          alert(errMessage);
       });
      }
```

.

☞ **중복체크 결과를 아이디 text박스 옆에 출력한다.**

```
// 중복체크 결과 값을 저장 할  idCheckResult - 중복입니다. or 사용가능합니다.
 const [idCheckResult ,setIdCheckResult] = useState("");


// 아이디 중복여부에 따른 css를 적용하기 위해 상태 변수 - true이면 중복, false이면 사용가능
 const [isCheckResult ,setIsCheckResult] = useState(false);
```

8253jang@daum.net
장희정

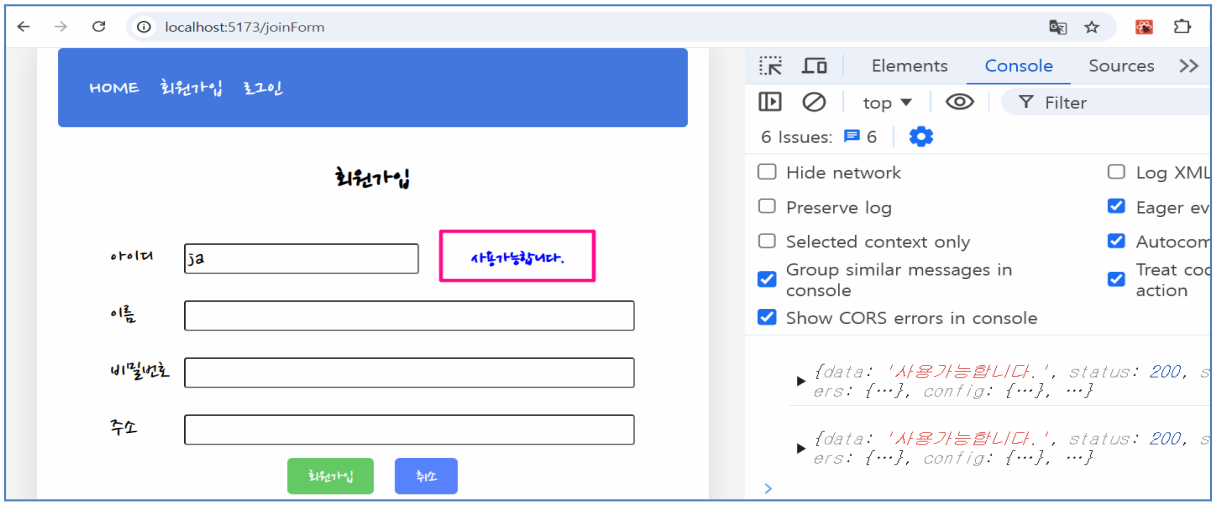
☞ **axios결과를 받아서** setIdCheckResult , setIsCheckResult 값변경

```
.then((res)=>{
    setIdCheckResult(res.data);

  res.data=="중복입니다." ?
            setIsCheckResult(true) :setIsCheckResult(false);
})
```

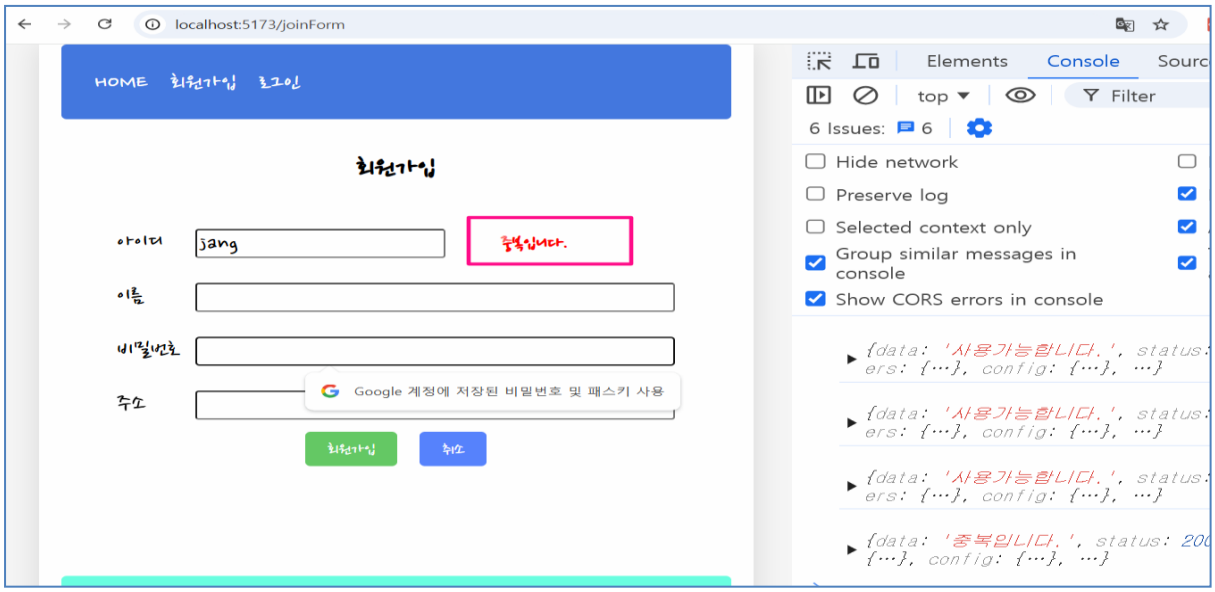
☞ **아이디 text박스 옆에 결과를 출력한다.**

```
<div className="idDuplicate">
  <LabelText text={"아이디"} name={"id"}   changeValue={changeValue}/>
 <div className="idText" style={ isCheckResult ? {color: "red"} : {color: "blue" } }>{idCheckResult}</div>
</div>
```

실행결과



8253jang@daum.net
장희정



☞ 회원가입 클릭

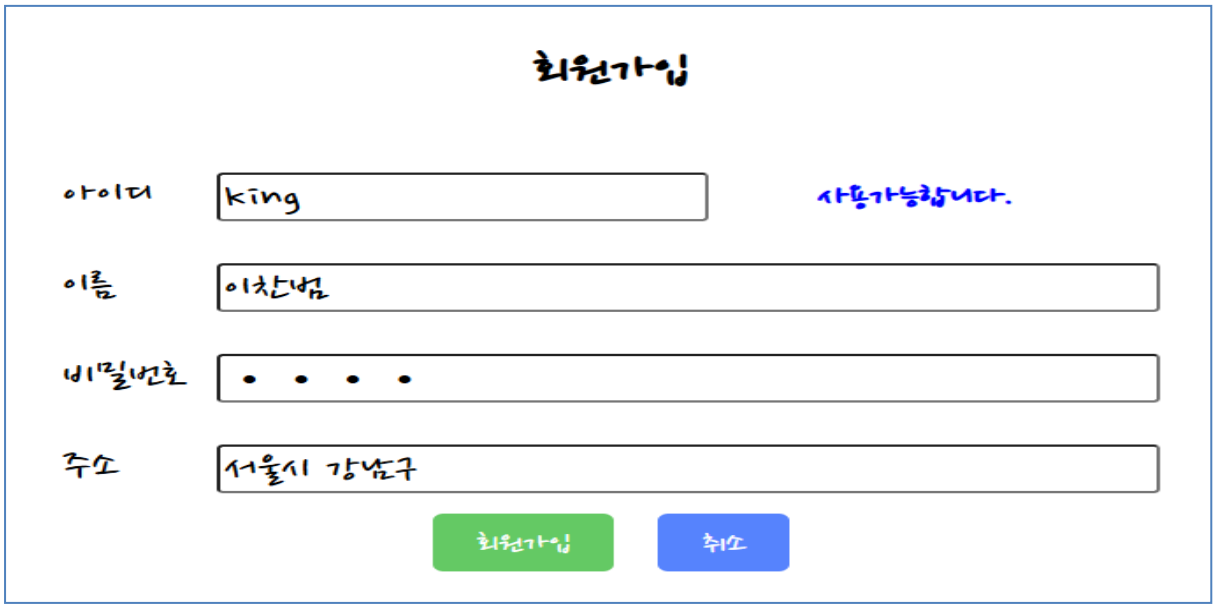
: 가입을 성공하면 "/" Home으로 이동한다.

: import{ useNavigate } from'react-router-dom';

```
const navigator = useNavigate();
//가입하기
const submitJoin = (e)=>{
    axios({
    method:"POST",
    url : "http://localhost:9000/members",
    data : member,
    })
    .then((res)=>{
    console.log(res);
    navigator("/")
    })
    .catch((err)=>{
        console.log(err)
        let errMessage = err.response.data.type +"\n";
        errMessage += err.response.data.title +"\n";
        errMessage += err.response.data.detail +"\n";
        errMessage += err.response.data.status +"\n";
        errMessage += err.response.data.instance +"\n";
        errMessage += err.response.data.timestamp;
        alert(errMessage);
    });

};
```

실행결과

8253jang@daum.net
장희정



로그인 하기

: backend spring security를 이용하여 인증을 진행 할 예정이다.
때문에 입력한 아이디와 비밀번호는 Controller가 아닌 Spring security의 filter에 전달된다.
Security는
Parameter 정보 -> 아이디 : username , 비번 : password
parameter정보를 json이 아닌 form data로 전송 해야한다.
요청주소 -> /login

☞ **로그인 폼 디자인 – LoginForm.jsx**

```
77      return (
78        <div className="LoginForm">
79          <h1 className="h1">로그인</h1>
80          <LabelText text={"아이디"} name={"username"}  changeValue={changeValue} />
81          <LabelText text={"비밀번호"} name={"password"} changeValue={changeValue} />
82
83          <div className="divBtn">
84              <Button text={"로그인"}  type={"button"}  onClick={submitLogin}/>
85              <Button text={"취소"}  type={"reset"}  onClick={cancelBack}/>
86          </div>
87
88        </div>
89      )
```

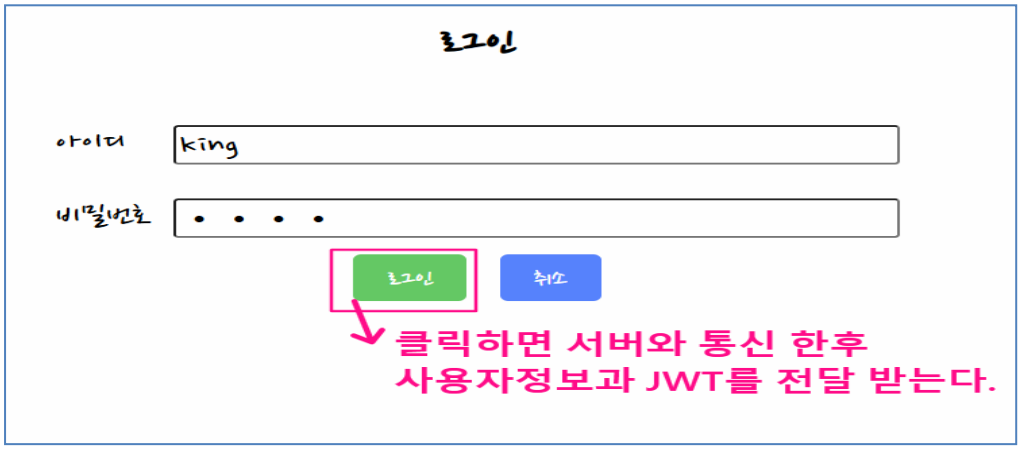
8253jang@daum.net
장희정

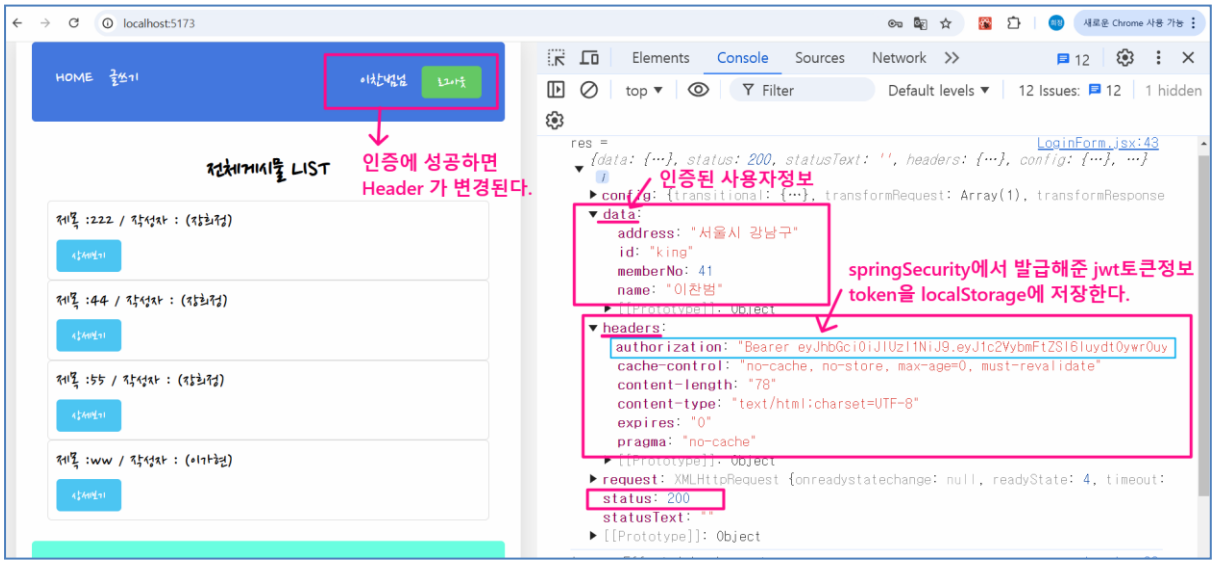
☞ **로그인 기능구현**

```
// 인증에필요한 username, password 상태관리를 위한 useState
const [ member, setMember ] = useState({
    username :'' ,
    password:'',
  });

// input 에 값이 입력될 때 상태값 수정
const changeValue = (e)=>{
    setMember({
      ...member,
        [e.target.name]:e.target.value}
    )
  }//

const navigator = useNavigate();

//로그인 버튼을 클릭했을 때 axios
  const submitLogin = (e)=>{
    e.preventDefault();//submit 이벤트막음

    const  formData = new FormData(); //폼전송으로보내기위한작업
    formData.append("username", member.username);
    formData.append("password", member.password);

    axios({
      method : "POST",
      url : "http://localhost:9000/login",
      data : formData,
    })
    .then((res)=>{
      console.log("res = " , res)
    })
    .catch((err)=>{
        alert("정보를 다시 확인해주세요.");
  }
```

8253jang@daum.net
장희정

실행결과





☞ 로그인이 성공하면 인증된 사용자의 정보를 1)localStorage에 저장하고 App.jsx에 있는 isLogin 변수를 true로 변경하기 위해 useContext로 Context에 있는 LoginContext를 가져와서 2) setIsLogin 함수를 호출한다.
3)Home.jsx로 이동한다.

```
//인증된 정보 localStorage 에 저장
 localStorage.setItem("memberNo", res.data.memberNo);
 localStorage.setItem("id", res.data.id);
 localStorage.setItem("name", res.data.name);
 localStorage.setItem("address", res.data.address);
 localStorage.setItem("Authorization", res.headers.authorization);
```

8253jang@daum.net
장희정

```
//App.js 에 있는 isLogin 변수를 true 변경한다.
setIsLogin(true);

//Home.jsx 로 이동한다.
 navigator("/");
```

App.jsx파일

먼저, App.jsx문서에서

로그인 여부를 체크 할

const [isLogin, setIsLogin] = useState(false); 선언

main.jsx 요청되면 -> App.jsx 로딩 ->useEffect()를 이용하여localStorage에 인증된 정보가 있는지 확인하여 상태변수 isLogin 을 변경한다.(있으면 true, 없으면 false)

1) createContext()를 이용하여 하위 컴포넌트들이 isLogin, setIsLogin 를 공유 하도록 한다.

```
13  import { createContext, useEffect, useState } from "react";
14
15  /*useContext 를 이용해서 하위 컴포넌트들이 데이터 공유하기*/
16  export const LoginContext = createContext();
```

```
13  import { createContext, useEffect, useState } from "react";
14
15  /*useContext 를 이용해서 하위 컴포넌트들이 데이터 공유하기*/
16  export const LoginContext = createContext();
17
18  function App() {
19    //로그인 여부를 판단하는 상태변수 (true이면 로그인, false 로그아웃)
20    const [isLogin, setIsLogin] = useState(false);
21
22    //컴포넌트가 mount or update 될때 로그인 여부에 따른 상태값 변경
23    useEffect(() => {
24      localStorage.getItem("id") != null ? setIsLogin(true) : setIsLogin(false);
25
26      console.log("App useEffect isLogin = ", isLogin);
27    },[isLogin]);
28
29
30    return (
31
32      <div className="App">
33        <LoginContext.Provider  value={ {isLogin:isLogin , setIsLogin:setIsLogin } }>
34          <Header/>
35          <main className="AppManin">
```

8253jang@daum.net
장희정

App.jsx소스

```
/*useContext 를 이용해서 하위 컴포넌트들이 데이터 공유하기*/
export const LoginContext = createContext();

function App() {
  //로그인 여부를 판단하는 상태변수 (true이면 로그인, false 로그아웃)
  const [isLogin, setIsLogin] = useState(false);

  //컴포넌트가 mount or update 될때 로그인 여부에 따른 상태값 변경
  useEffect(() => {
    localStorage.getItem("id") != null ? setIsLogin(true) : setIsLogin(false);

    console.log("App useEffect isLogin = ", isLogin);
  },[isLogin]);

  return (

    <div className="App">
      <LoginContext.Provider   value={ {isLogin:isLogin , setIsLogin:setIsLogin } }>
       <Header/>
       <main className="AppManin">
        <Routes>
            <Route path='/' element={<Home/>} />
            <Route path='/saveForm' element={<SaveForm/>} />
            <Route path='/board/:id' element={<Detail/>} />
            <Route path='/updateForm/:id' element={<UpdateForm/>} />
            <Route path='/loginForm' element={<LoginForm/>} />
            <Route path='/joinForm' element={<JoinForm/>} />
            <Route path='/*' element={<NotFound/>} />
        </Routes>
     </main>
        <Footer />
      </LoginContext.Provider>
    </div>

  )
}

export default App
```

Header.jsx파일

8253jang@daum.net
장희정

```
6
7   const Header =()=>{
8     const {isLogin , setIsLogin} = useContext(LoginContext);
9
10    const navigator = useNavigate();
11    const logoutCheck = ()=>{
12      localStorage.clear();//모든 localStorage의 정보 지우기
13
14      setIsLogin(false);
15
16      navigator("/");
17    }
18
19      return (
20        <>
21          <div className="Header">
22              <Link to="/" >HOME</Link>
23          1){ isLogin && <Link to="/saveForm">글쓰기</Link>}
24
25          2){!isLogin && <Link to="/joinForm">회원가입</Link>}
26
27          3){isLogin ? (
28              <div className="login">
29               <span>{localStorage.getItem("name")}님</span>
30               <Button text={"로그아웃"} type={"button"} onClick={logoutCheck} />
31              </div>
32           )
33           :
34              <Link to="/loginForm">로그인</Link>
35           }
36
37          </div>
38        </>
39      )
```

Header.jsx소스

```
const Header =()=>{
  const {isLogin , setIsLogin} = useContext(LoginContext);

  const navigator = useNavigate();
  const logoutCheck = ()=>{

    localStorage.clear();//모든 localStorage의 정보 지우기

    setIsLogin(false);

    navigator("/");
  }
    return (
      <>
        <div className="Header">
            <Link to="/" >HOME</Link>
            { isLogin && <Link to="/saveForm">글쓰기</Link>}

            {!isLogin && <Link to="/joinForm">회원가입</Link>}
```

```
            {isLogin ? (
                <div className="login">
                  <span>{localStorage.getItem("name")}님</span>
                  <Button text={"로그아웃"} type={"button"} onClick={logoutCheck} />
                </div>
            )
            :
               <Link to="/loginForm">로그인</Link>
           }

        </div>
      </>
    )
  }

  export default Header;
```

인증 안 된 경우

-인증 된 경우

☞LoginForm.jsx 수정

: useContext를 이용하여 전달된 Context를 사용한다

```
const { setIsLogin} = useContext(LoginContext);
```

☞로그인이 성공했을 때 콜백함수에서 인증여부 상태변수의 값을 변경한다.

```
//App.js 에 있는 isLogin 변수를 true 변경한다.
   setIsLogin(true);
```

그리고, useNavigate()를 이용하여 "/" Home으로 이동한다.

```
//페이지 이동하는 방법
const navigator = useNavigate();

//Home.jsx 로 이동한다.
 navigator("/");
```

☞Header.jsx 문서에서 useCcontext 선언

```
const {isLogin , setIsLogin} = useContext(LoginContext);
```

☞Header.jsx 문서에서 로그아웃 되었을 때 이벤트
: 모든 Storage에 저장된 정보를 삭제한 후
setIsLogin**함수에 false를 설정한다.**
그리고 , /로 이동한다.

```
const navigator = useNavigate();
 const logoutCheck = ()=>{
   localStorage.clear();//모든 localStorage 의 정보 지우기
  setIsLogin(false); //App.jsx 에 선언된 state 가 변경되면서 Header 컴포넌트 리랜더링
   navigator("/");
 }
```

☞Header.jsx 문서에서 인증여부에 따른 메뉴 출력

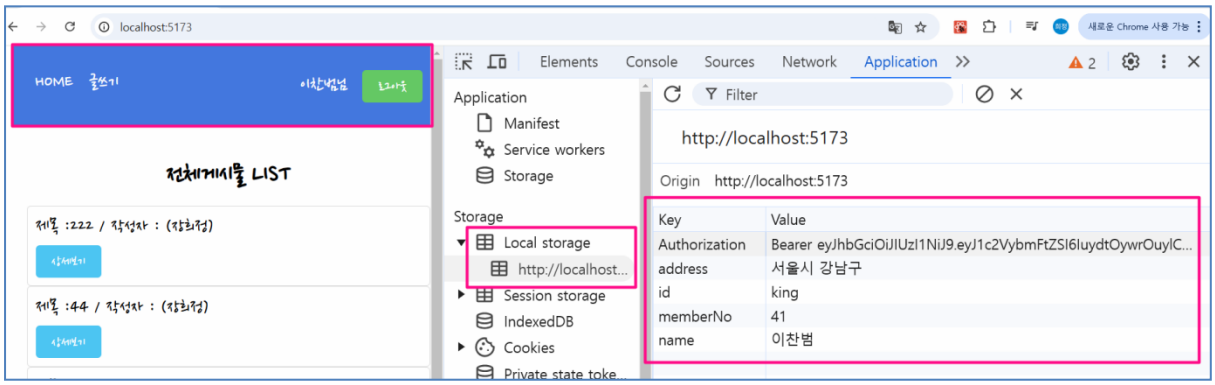
```
return (
      <>
        <div className="Header">
            <Link to="/" >HOME</Link>
            { isLogin && <Link to="/saveForm">글쓰기</Link>}

            { !isLogin && <Link to="/joinForm">회원가입</Link>}

            { isLogin ? (
                <div className="login">
                 <span>{ localStorage.getItem("name") }님</span>
                 <Button text={"로그아웃"} type={"button"} onClick={logoutCheck} />
```

8253jang@daum.net
장희정

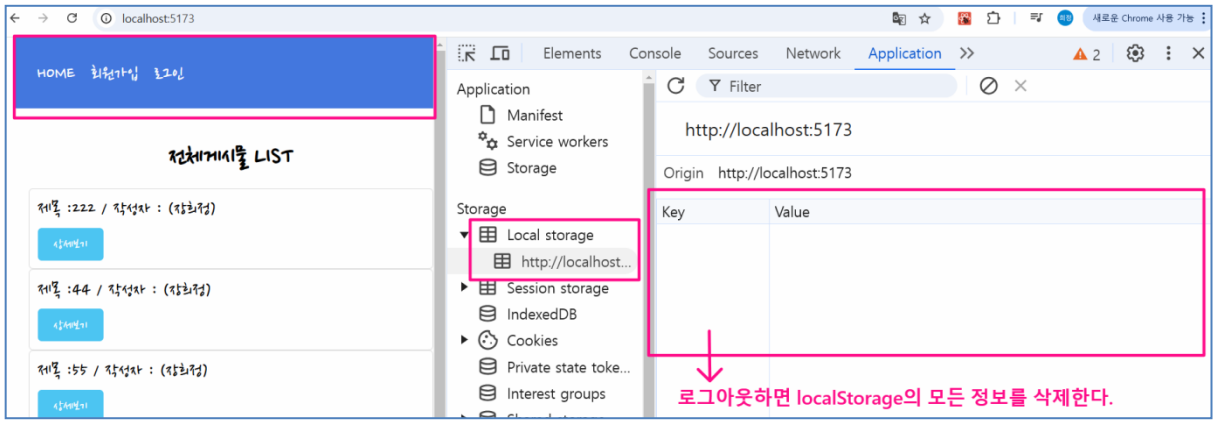
```
                </div>
            )
            :
                <Link to="/loginForm">로그인</Link>
            }

        </div>
    </>
)
```

실행결과

로그인성공 했을 때

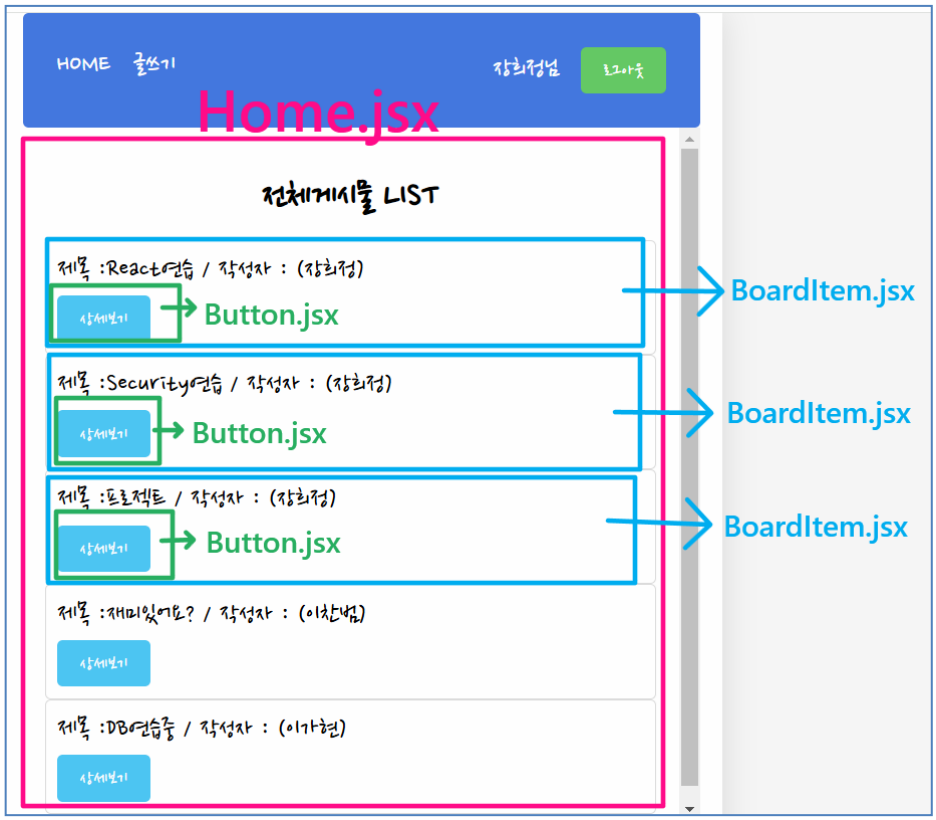


로그아웃 했을 때



☞ 전체 게시물 보기

8253jang@daum.net
장희정



src> components > Button.jsx

```jsx
import './Button.css';

/*
props정보
   text:글씨 ,
   type:버튼유형  ,
   onClick:함수 ,
   detail : 상세보기 버튼인경우 디자인 다르게하기 위한 속성
*/
const Button = ({text, type,  onClick , detail })=>{

    return (
        <button  type={type} className={`Button Button_${type} ${detail}`}
         onClick={onClick}  >{text}</button>
    )
}

export default Button;
```

src> components > Button.css

8253jang@daum.net
장희정

src > components > # Button.css > .Button_reset

```
.Button{
    background-color: rgb(236, 236, 236);
    cursor: pointer;
    border: none;
    border-radius: 5px;
    padding: 10px 20px;
    font-size: 18px;
    white-space: nowrap;/*이버튼안에 있는 text가 화면이 줄어들어도 줄바꿈되지않도록 한다.*/
}

.Button_button{
    background-color: rgb(100, 201, 100);
    color: white;
  }

  .Button_reset{
      background-color: rgb(86, 131, 253);
      color: white;
  }

  .detail{
    background-color: rgb(76, 198, 243);
    color: white;
  }
```

src> pages > Home.jsx(전체소스)

```
const Home =()=>{
  const { handleLoginChange} = useContext(LoginContext);

   //DB 목록을 저장해서 관리 할 useState
   const [boards , setBoards] = useState([]);

   const nav = useNavigate();

     //서버에서 데이터 조회    - axios
     useEffect(()=>{
         axios
         .get("http://localhost:9000/boards", {
         headers: {
         Authorization : localStorage.getItem("Authorization"),
         },})
         .then((res)=>{
              //console.log(res);
             setBoards(res.data);//state 에서 관리되는 변수가 변경되어 update(re-rendering)
         })
         .catch((err)=>{
              console.log(err)
```

```
            if(err.response.status===403){
                localStorage.clear();//모든 세션의 정보 지우기
                handleLoginChange(false);
                alert("로그인하고 이용해주세요.");
                nav("/loginForm");
            }else{
                alert(err.response.data.detail) ;
            }
        });
    }, []);

    return (
        <div className="Home">
         <h1>전체게시물 LIST</h1>
          {
            boards.map( (board)=> <BoardItem key={board.id} board={board}/> )
          }
        </div>
    )
}

export default Home;
```

실행결과

Home.jsx에 코드를 추가 해보면서 코드을 완성해보자.

8253jang@daum.net
장희정

BoardItem 컴포넌투는 데이터베이스에 있는 게시물의 개수 만큼 반복되야한다.(map함수이용)
그러기 위해서는 DB 테이블에 있는 레코드를 조회 해야 한다.
Home.jsx 에서 React Hook의 종류인 useEffect를 이용하여 DB에 들어있는 데이터를 비 동기통신으로(axios) 조회한다.
([ ] 안넣는다면?) =>최초실행 + 상태값변경 때마다 실행됨

☞axios를 이용하여 모든 게시물 조회

- Home.jsx import

```
import { useContext, useEffect, useState } from "react";
import BoardItem from "../../components/BoardItem";
import "./Home.css";
import axios from 'axios';
import { useNavigate } from "react-router-dom";
import { LoginContext } from "../../App";
```

-Home.jsx 마운트(최초로딩) 되었을 때 만 backend에서 데이터 가져오기
spring에서 spring security + jwt로 구현되어 있어 headers에 Authorization 에 해당하는 jwt 토큰을 함께 요청한다.

```
//서버에서 데이터 조회    - axios
    useEffect(()=>{
        axios
        .get("http://localhost:9000/boards", {
        headers: {
        Authorization: localStorage.getItem("Authorization"),
        },})
        .then((res)=>{
            //console.log(res);
          setBoards(res.data);//state 에서 관리되는 변수가 변경되어 update(re-rendering)
        })
        .catch((err)=>{
            console.log(err)
            if(err.response.status===403){
                localStorage.clear();//모든 세션의 정보 지우기

                handleLoginChange(false);
```

```
                alert("로그인하고 이용해주세요.");
                nav("/loginForm");
            }else{
                alert(err.response.data.detail) ;
            }
        });
    }, []);   //([ ] 안넣는다면?) =>최초실행 + 상태값변경 때마다 실행됨
```

-Home.jsxDB에서 조회된 boards의 정보를 map을 이용하여출력한다.
BoardItem 컴포넌트에 props를 이용하여 board의 정보를 전달한다.

```
return (
    <div className="Home">
     <h1>전체게시물 LIST</h1>
      {
        boards.map( (board)=> <BoardItem key={board.id} board={board}/> )
      }
    </div>
  )
```

-BoardItem.jsx 에서 props로 전달된 데이터를 받아서 출력

```
1  import { Link, useNavigate } from "react-router-dom";
2  import "./BoardItem.css";
3  import Button from "./Button";
4  const BoardItem =( {board} )=>{
5    //console.log(board);
6
7    const nav= useNavigate();
8    const check = ()=>{
9        if(localStorage.getItem("id")){
10           nav("/board/"+board.id)
11       }else{
12           alert("로그인하고 이용해주세요.");
13           nav("/loginForm")
14       }
15   }
16   return (
17     <div className="BoardItem">
18       <div className="title_writer">
19         제목 :{board.title}  / 작성자 : ({board.member.name})
20         </div>
21       <div  className="detailLink">
22         <Button  text={"상세보기"} type={"button"} onClick={check} detail={"detail"}/>
23       </div>
24     </div>
25   )
26 }
```

8253jang@daum.net
장희정

☞ 글 쓰기 화면구성(SaveForm.jsx)

```jsx
return (
  <>
  <h1 className="h1">  게시물 등록하기 </h1>

    <LabelText text={"title"} name={"title"} changeValue={changeValue}/>
    <LabelText text={"writer"} name={"name"} readOnly={"yes"}/>

    <div className="divContent">
      <div className="divContent_label">cotent</div>
      <textarea className="divContent_textarea" placeholder="오늘은 어땠나요?"
      name="content" onChange={changeValue}></textarea>
    </div>

    <div className="divBtn">
        <Button text={"등록"}  type={"button"}  onClick={submitBoard}/>
        <Button text={"취소"}  type={"reset"}  onClick={()=>navigator(-1)}/>
    </div>
  </>
)
```

인증된 사용자만이 글을 등록할수 있다.
때문에, 작성자는 읽기전용으로 하고 localStorage에서 조회하여 출력해준다.

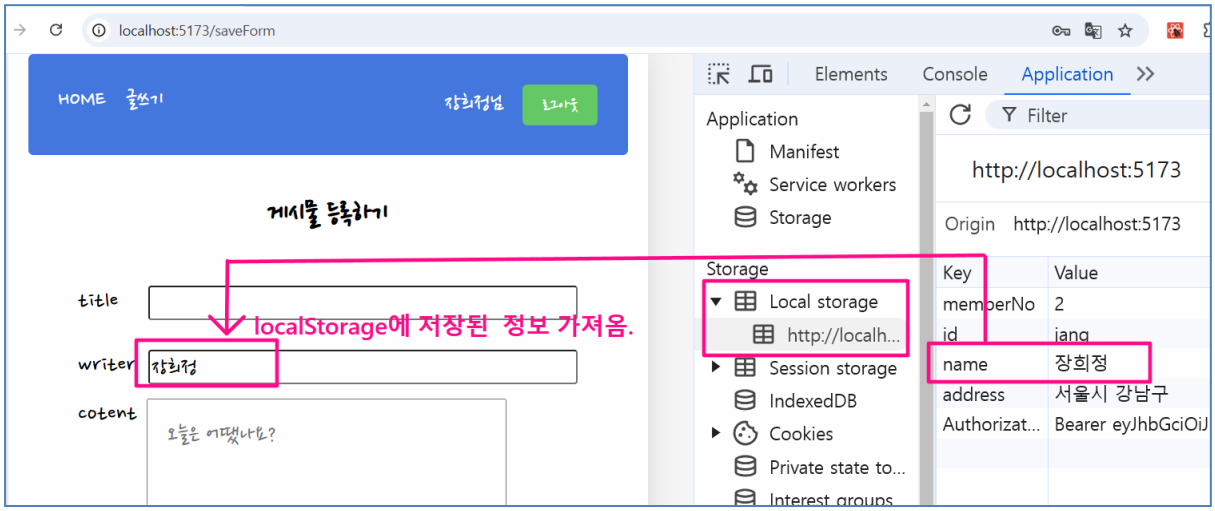
뒤로가기

LabelText.jsx파일

```jsx
const LabelText =({text, name, initValue , changeValue , readOnly})=>{
  const [value, setValue] = useState(initValue || ""); //initValue값 falsy(undefined,null,NaN,false,0)이면 ""로 초기값설정

  useEffect(()=>{
    readOnly && setValue(localStorage.getItem("name"));
  });
```

```jsx
<input
className={`${initValue ?  'input_disabled':'input'}`}
type={`${(name=="password" || name=="pwd") ? "password" : "text"}`}
 name={name}
value={value}
readOnly={readOnly ? true : undefined}
onChange={onChangeCheck} />
```

실행화면



8253jang@daum.net
장희정

- text박스에 값이 입력될 때 state를 변경하기

: 작성자의 pk는 localStorage에서 조회

```
const SaveForm =()=>{
 //게시물 상태관리
 const [board, setBoard] = useState({
    title: "",
    content: "",
    memberNo: localStorage.getItem("memberNo"),
 }); //제목,내용, 작성자 pk=memberNo,

 //input 에 값이 변경될 때
 const changeValue = (e) => {
    setBoard({
       ...board,
       [e.target.name] : e.target.value,
    });
 };
```

- 모든 값들을 입력 한 후 axios로 비동기 통신

: 등록이 성공하면 "/" 로 이동

: import{ useNavigate } from"react-router-dom";

: importaxiosfrom"axios";

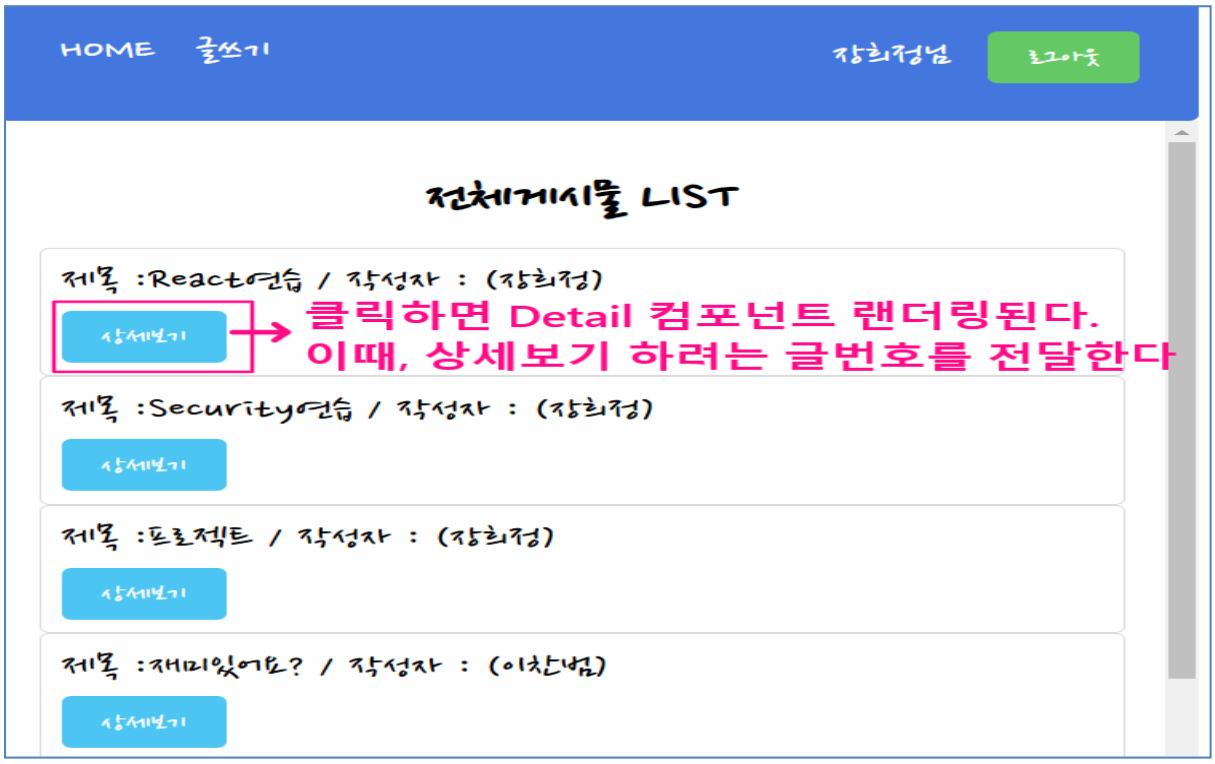
```
//페이지 이동하는 방법
 const navigator = useNavigate();

//등록하기 클릭
 const submitBoard = (e) => {
    axios({
       method: "POST",
       url: "http://localhost:9000/boards/board",
       data: board,
       headers: {
          Authorization: localStorage.getItem("Authorization"),
          }
    })
       .then((res) => {
```

```
        console.log(res);
        navigator("/");
      })
      .catch((err) => {
        if (err.response.status === 403) {
          alert("로그인된 사용자만이 글을 등록 할 수 있어요.");
          navigator("/");
        } else {
         alert(err.response.data.detail);
        }
    });
};
```

실행화면

8253jang@daum.net
장희정

☞ **BoardItem.jsx 파일에서 상세보기 클릭**



Home.jsx에서

☞**Detail.jsx 의 컴포넌트가 호출되면**

- 전달된 파라미터를 **useParams**로 받는다.
- 게시물에 대한 정보를 저장할 **useState**를 선언한다.
- **useEffect 함수에서** 최초의 마운트 되었을 때 axios를 이용하여 글번호에 해당하는 게시물 조회를 서버에 요청 한 후 게시물의 정보를 응답 받는다.

- 응답결과가 error이면 에러메시지를 출력하고 **useNavigate()를**이용하여 "/"로 이동한다.

8253jang@daum.net
장희정

```
4   const BoardItem =( {board} )=>{
5     //console.log(board);
6
7     const nav= useNavigate();
8     const check = ()=>{
9       if(localStorage.getItem("id")){
10        nav("/board/"+board.id)
11      }else{
12        alert("로그인하고 이용해주세요.");
13        nav("/loginForm")
14      }
15    }
16    return (
17      <div className="BoardItem">
18        <div className="title_writer">
19          제목 :{board.title}  / 작성자 : ({board.member.name})
20          </div>
21        <div  className="detailLink">
22          <Button  text={"상세보기"} type={"button"} onClick={check} detail={"detail"}/>
23        </div>
24      </div>
```

Detail.jsx 컴포넌트

-Detail.jsx파일 일부 소스

```
const Detail =()=>{

  //파라미터를 받는다.
  const { id } = useParams(); //  nav("/board/"+board.id) 요청에서 id 가져오기

  const [board, setBoard] = useState({
    id: "",
    title: "",
    content: "",
    member: {},
    regDate:""
  });

 const nav = useNavigate();

 useEffect(() => {
   axios
     .get("http://localhost:9000/boards/" + id, {
       headers: {
         Authorization: localStorage.getItem("Authorization"),
       },
     })
     .then((res) => {
       console.log(res)
```

```
        setBoard(res.data);
      })
      .catch((err) => {
          errFun(err);
      });
}, []);

//에러 출력 함수
const errFun = (err) => {
  if (err.response.status === 403) {
    alert("로그인하고 이용해주세요.");
  } else {
    let errMessage = "오류 = " + err.response.data.type + "₩n";
    errMessage += err.response.data.title + "₩n";
    errMessage += err.response.data.status + "₩n";
    errMessage += err.response.data.instance + "₩n";
    errMessage += err.response.data.timestamp;
    alert(errMessage);
  }

  nav("/");
};

 return (
    <div className="Detail">
     <h1>상세보기</h1>

      <LabelText text={"글번호"}  initValue={board.id}/>
      <LabelText text={"제목"}  initValue={board.title}/>
      <LabelText text={"작성자"}  initValue={board.member.name}/>
      <LabelText text={"등록일"}  initValue={board.regDate}/>

      <div className="divContent">
        <div className="divContent_label">cotent</div>
        <textarea className="divContent_textarea" placeholder="오늘은 어땠나요?"
         name="content"  disabled value={board.content}></textarea>
     </div>

    { board.member.name === localStorage.getItem("name")  &&
     <div className="divBtn">
          <Button text={"수정"}  type={"button"}  onClick={()=>{nav("/updateForm/"+id)}}/>
          <Button text={"삭제"}  type={"button"} onClick={deleteBoard} />
      </div>
```

```
      }

      </div>
    )
  }

  export default Detail;
```

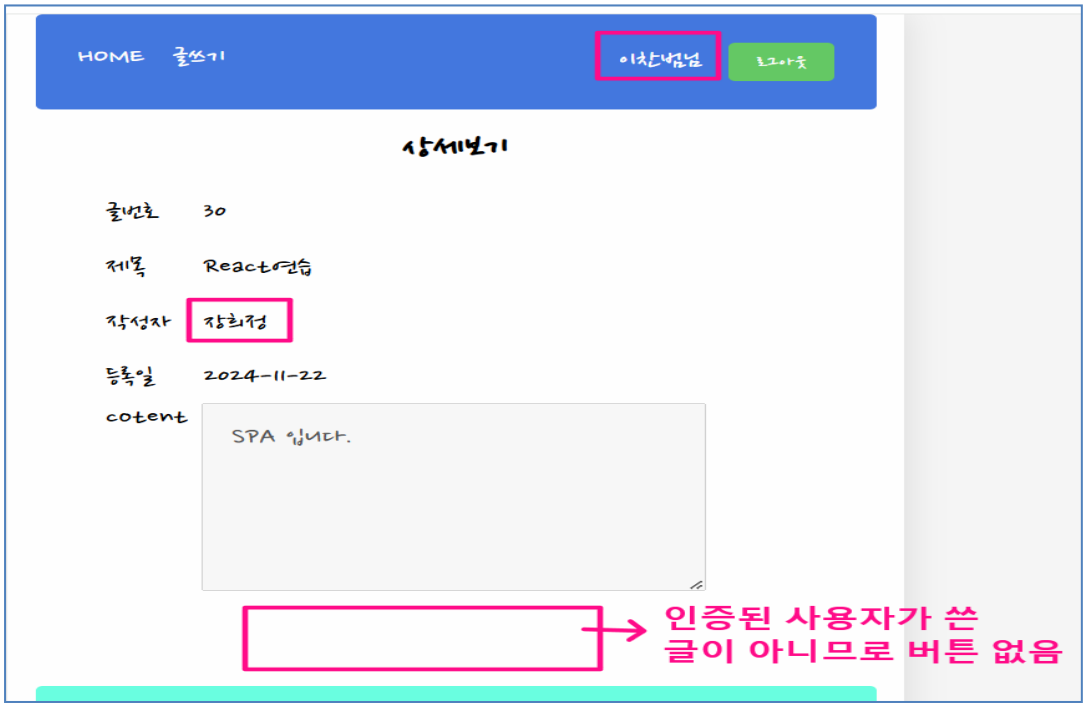
src> components > LabelText.jsx

```
/*
  props정보
    text : label글씨
    name : input 요소 name값
    initValue : input요소의 value 초기값
    changeValue : input 값이 onChange될때 호출 될 함수
    readOnly : inputt요소 readOnly 적용
*/
const LabelText ={{text, name, initValue , changeValue , readOnly}}=>{
    const [value, setValue] = useState(initValue || ""); //initValue값 falsy(undefined,null,NaN,false,0)이면 ""로 초기값설정

     useEffect(()=>{
      readOnly && setValue(localStorage.getItem("name"));
     });

    const onChangeCheck = (e)=>{
      setValue(e.target.value)
      changeValue(e);
    }
```

```
    return (
        <div className="LabelText" >
            <label className="label">{text}</label>

            {
             initValue ?  <div>{initValue}</div> :

            <input
            className={`${initValue ?  'input_disabled':'input'}`}
            type={`${(name==="password" || name==="pwd") ? "password" : "text"}`}
             name={name}
            value={value}
            readOnly={readOnly ? true : undefined}
            onChange={onChangeCheck} />
          }
        </div>
    )
```

- Detail.jsx자신이 쓴 글 클릭하면, 수정, 삭제 버튼생성

-Detail.jsx자신이 쓴 글이 이나면 수정, 삭제 버튼 없음.



-수정 and 삭제 이벤트

```
//삭제 클릭
const deleteBoard = (id)=>{
```

```
}

//수정클릭
constupdateBoard = ()=>{

}
```

☞ **삭제하기**

: 삭제를 클릭하면 글번호를 인수로 받아 서버에 전송한다.
이때 headers를 이용하여 token을 함께 전송한다.

```
headers: {
        Authorization: localStorage.getItem("Authorization"),
}
```

: 삭제가 성공하면 "/" 으로 이동한다.

```
const deleteBoard = () => {
    axios({
        method:"DELETE",
        headers: {
        Authorization: localStorage.getItem("Authorization"),
        },
        url : "http://localhost:9000/boards/"+id,
        })
        .then((res)=>{
        if(res.data ==="ok")
            nav("/") ;
        else
         alert("삭제되지 않았습니다.");
     })
    .catch((err)=>{
         console.log("err = " + err)
         errFun(err);
    });
};//삭제끝
```

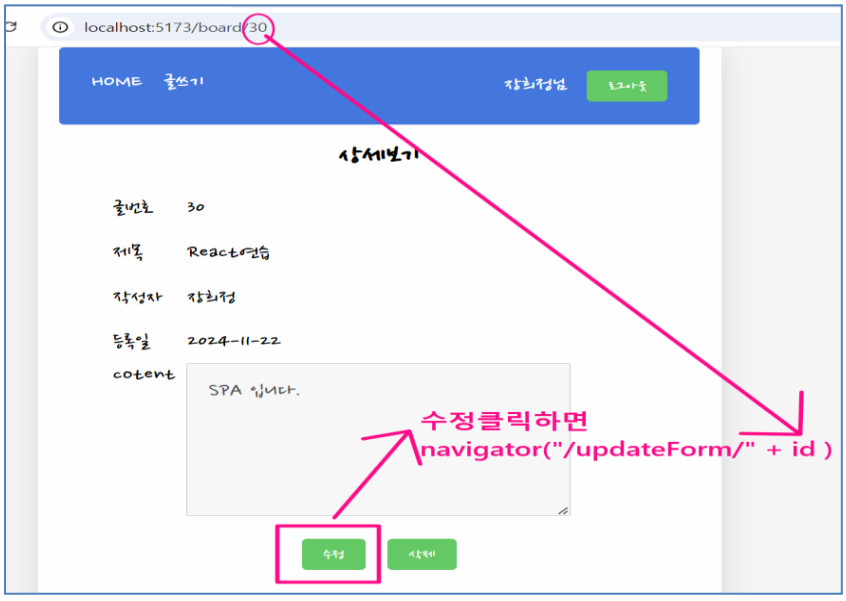
8253jang@daum.net
장희정

☞ **수정하기**

: 수정을 클릭하면 UpdateForm.jsx컴포넌트로 이동한다.
이때 수정하려는 게시물의 글번호를 전달한다.

```
//수정하기
constupdateBoard = ()=>{
    navigator("/updateForm/"+id);
}
```

실행화면



UpdateForm.jsx 실행화면

8253jang@daum.net
장희정

☞ **UpdateForm.jsx에서**

: usePramas()를 이용하여 전달된 글번호를 받는다.

: useState()를 이용하여 게시물의 정보를 저장할 상태변수를 선언한다.

: useEffect()를 이용하여 컴포넌트가 마운트(로딩)되었을 때 서버와 통신하여 게시물의 정보를 가져온다.

headers에 Authorization:localStorage.getItem("Authorization")전송

-import 문

```
import { useNavigate, useParams } from "react-router-dom";
import Button from "../../components/Button";
import LabelText from "../../components/LabelText";
import './UpdateForm.css';
import { useEffect, useState } from "react";
import axios from "axios";
```

```
const { id } = useParams(); //

const [board, setBoard] = useState({
```

```
    title: "",
    content: "",
    member: {},
});

//페이지 이동하는 방법
const navigator = useNavigate();

useEffect(() => {
    axios
        .get("http://localhost:9000/boards/" + id, {
            headers: {
                Authorization: localStorage.getItem("Authorization"),
            },
        })
        .then((res) => {
            setBoard(res.data);
        })
        .catch((err) => {
            errFun(err);
        });
}, []);

const errFun = (err) => {
    if (err.response.status === 403) {
        alert("로그인하고 이용해주세요.");
    } else {
        let errMessage = "오류 = " + err.response.data.type + "₩n";
        errMessage += err.response.data.title + "₩n";
        errMessage += err.response.data.status + "₩n";
        errMessage += err.response.data.instance + "₩n";
        errMessage += err.response.data.timestamp;
        alert(errMessage);
    }
    navigator("/");
};
```

8253jang@daum.net
장희정

- 수정 폼

```
return (
      <>
       <h1 className="h1">   수정하기  </h1>
      <form>
        <LabelText text={"title"} name={"title"} initValue={board.title}
        changeValue={changeValue} />

        <LabelText text={"writer"} name={"name"}   readOnly="yes"/>

        <div className="divContent">
          <div className="divContent_label">cotent</div>
          <textarea
          className="divContent_textarea"
          placeholder="오늘은  어땠나요?"
           name="content"
           value={board.content}
           onChange={changeValue}></textarea>
       </div>

        <div className="divBtn">
             <Button text={"수정"}   type={"button"}   onClick={submitBoard}/>
             <Button text={"취소"}   type={"reset"} />
        </div>
        </form>
      </>
    )
```

```
constchangeValue = (e)=>{

 }


 constsubmitBoard = (e)=>{

  };
```

- text박스에 값이 입력될 때 board의 정보를 수정한다.

```
  const changeValue = (e)=>{
    setBoard(
        {...board ,
         [e.target.name] : e.target.value
    })
 }
```

- 수정을 클릭하면 서버에 수정을 요청하고 성공하면 navigator("/boards/"+id); 로 이동한다.

```
const submitBoard = (e)=>{
    axios({
        method:"PUT",
        url : "http://localhost:9000/boards/"+id,
        data : board,
        headers: {
        Authorization: localStorage.getItem("Authorization"),
        }
        })
        .then((res)=>{
            navigator("/board/"+id);
        })
        .catch((err)=>{
            errFun(err);
        });
};
```

SpringBoot Entity 생성

☞ **Member.java**

```
@Entity
@Getter
@Setter
@Builder
@NoArgsConstructor
```

```
@AllArgsConstructor
public class Member {
@Id
private String   id;
private String   pwd;
private String   name;
private String   address;

@CreationTimestamp
private LocalDateTimeregDate;
}
```

Table 생성 결과

```
desc Member;
```

질의 결과 × | 질의 결과 1 × | 질의 결과 2 × | 질의 결과 3 ×

작업이 완료되었습니다.(1.499초)

```
이름          널?        유형
--------- -------- ------------------
REG_DATE           TIMESTAMP(6)
ADDRESS            VARCHAR2(255 CHAR)
ID        NOT NULL VARCHAR2(255 CHAR)
NAME               VARCHAR2(255 CHAR)
PWD                VARCHAR2(255 CHAR)
```

☞ **Board.java**

```
@AllArgsConstructor
@NoArgsConstructor
@Setter
@Getter
@ToString
@Entity //서버 실행시에 해당 객체로 테이블 매핑생성
@Builder
public class Board {
```

```
@Id//pk 를 해당 필드로 한다
//@GeneratedValue(strategy = GenerationType.IDENTITY)//해당 디비 번호증가 전략을
따라가겠다.
    //@GeneratedValue(strategy = GenerationType.AUTO)//해당 디비 번호증가 전략을
따라가겠다.
@GeneratedValue(strategy = GenerationType.SEQUENCE,generator = "board_id")
@SequenceGenerator(allocationSize = 1, sequenceName = "board_id" , name = "board_id")
private Long id;//글번호

private String title;//제목

@Column(length =100)
private String content;//내용

@ManyToOne
//@JoinColumn(name ="member_id")
    //@ManyToOne(fetch = FetchType.LAZY)
private Member member;//작성자

@CreationTimestamp
private LocalDateTimeregDate;//등록일

@UpdateTimestamp
private LocalDateTimeupdateDate;//수정일
}
```

Table결과

```
desc Board;

질의 결과 × | 질의 결과 1 × | 질의 결과 2 × | 질의 결과 3 ×
작업이 완료되었습니다.(1.029초)

이름            널?        유형
------------ -------- ------------------
ID           NOT NULL NUMBER(19)
REG_DATE              TIMESTAMP(6)
UPDATE_DATE           TIMESTAMP(6)
CONTENT               VARCHAR2(100 CHAR)
MEMBER_ID             VARCHAR2(255 CHAR)
TITLE                 VARCHAR2(255 CHAR)
```

☞ **MemberRepository.java**

```
2 usages
public interface MemberRepository extends JpaRepository<Member, String>{

}
```

☞ **MemberRepository.java**

```
2 usages
public interface BoardRepository extends JpaRepository<Board, Long>{
    //@Query("select distinct b from Board b join fetch b.member")
    no usages
    @Query("select  b from Board b join fetch b.member")
     List<Board> join();
}
```

☞**BoardReq.java**

```
@Getter
@Setter
@AllArgsConstructor
@NoArgsConstructor
@Builder
public class BoardRes{
private Long id;//글번호
private String title;//제목
```

```
private String content;//내용
private MemberResmember;//작성자
private String regDate;//등록일

public BoardRes(Board board) {
id = board.getId();
title=board.getTitle();
content=board.getContent();
regDate=board.getRegDate().toString();
member=new MemberRes(board.getMember().getId() , board.getMember().getName());

    }

/*public BoardRestoBoardRes(Board board){
        return BoardRes.builder()
                .id(board.getId())
                .title(board.getTitle())
                .content(board.getContent())
                .regDate(board.getRegDate().toString())
                .member(new MemberRes(board.getMember().getId() ,
board.getMember().getName()))
                .build();
   }*/
}
```

☞**BoardRes.java**

```
@Setter
@Getter
@ToString
public class BoardReq{
private String title;//제목
private String content;//내용
private String memberId;//작성자

public Board toBoard(BoardReqboardReq){
return Board.builder().title(boardReq.getTitle()).content(boardReq.getContent())
                .member(Member.builder().id(boardReq.memberId).build())
                .build();
```

```
    }
}
```

☞**MemberRes.java**

```
@Setter
@Getter
@AllArgsConstructor
@NoArgsConstructor
public class MemberRes{
    private String   id;
    private String   name;
}
```

☞**MemberService.java**

```
public interface MemberService{
/**가입 */
void   signUp(Member member);

String duplicateCheck(String id);

/**로그인*/
Member   signIn(String id, String pwd);
}
```

☞**MemberServiceImpl.java**

```
@Service
@RequiredArgsConstructor
public class MemberServiceImplimplements   MemberService{
private final MemberRepositorymemberRepository;

@Transactional
    @Override
public void signUp(Member member) {
Member m = memberRepository.save(member); //동일한 id가 들어오면 수정됨
System.out.println("m = " + m);
    }

@Transactional(readOnly = true)
```

```
@Override
public String duplicateCheck(String id) {
Member member= memberRepository.findById(id).orElse(null) ;
System.out.println("member = " + member);
if(member==null) return "사용가능합니다.";
else return "중복입니다.";

    }

@Transactional(readOnly = true)
@Override
public Member signIn(String id, String   pwd) {
Member member= memberRepository.findById(id).orElseThrow(
                    ()->new MemberAuthenticationException("아이디를 다시 확인해주세요.","Wrong
Id"));
if(!member.getPwd().equals(pwd)){
throw new MemberAuthenticationException("비밀번호를 확인해주세요.", "Wrong Pass");
        }

return member;
    }
}
```

☞**MemberRes.java**

☞**MemberRes.java**

☞Spring Security + JWT + React 연동 CORS 설정

: SecurityConfig.java 문서의 filterChain 메소드안에 추가

```
//CORS 설정
http
.cors((corsCustomizer->corsCustomizer.configurationSource(new CorsConfigurationSource() {
@Override
public CorsConfigurationgetCorsConfiguration(HttpServletRequest request) {

CorsConfiguration configuration = new CorsConfiguration();
```

```
configuration.setAllowedOrigins(Collections.singletonList("http://localhost:3000"));
configuration.setAllowedMethods(Collections.singletonList("*"));
configuration.setAllowCredentials(true);
configuration.setAllowedHeaders(Collections.singletonList("*"));
configuration.setMaxAge(3600L);

configuration.setExposedHeaders(Collections.singletonList("Authorization"));

return configuration;
                }

            }));
```

WebMvcConfig.java 작성

```
package web.mvc.cofig;

import org.springframework.context.annotation.Configuration;
import org.springframework.web.servlet.config.annotation.CorsRegistry;
import org.springframework.web.servlet.config.annotation.EnableWebMvc;
import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;

/**
 * WebMvcConfigurer 를 이용해서 @CrossOrigin 글로벌 설정
* */
@Configuration
@EnableWebMvc
public class WebMvcConfigimplements WebMvcConfigurer{
@Override
public void addCorsMappings(CorsRegistry registry) {
registry.addMapping("/**")
                    .allowedOrigins("http://localhost:3000")
                    .allowedMethods("OPTIONS","GET","POST","PUT","DELETE");
    }
}
```